라즈베리파이
중간고사 요약본

챕터별 정리

01 [1주차 요약] AIoT 시스템과 라즈베리파이

02 [2주차 요약] 무선 통신 가능한 라즈베리파이 만들기

03 [3주차 요약] 라즈베리파이 운영체제 (리눅스 기초 & 핵심 명령어)

04 [4주차 요약] 파이썬 기초 및 라즈베리파이 실습 핵심 정리

05 [5주차 라즈베리파이 카메라 활용]

06 [6장 라즈베리파이로 웹서버 만들기 - 핵심 요약]

07 [7장 메시지 통신과 MQTT - 핵심 요약]

CHAPTER 01

[1주차 요약] AIoT 시스템과 라즈베리파이

1. AIoT 개념

- IoT (사물인터넷): 사물 간 인터넷 연결을 통한 데이터 수집 및 네트워크 형성
- AIoT (AI + IoT): IoT로 수집한 데이터를 AI 모델로 학습·분석하여 능동적으로 제어

2. 아두이노 vs 라즈베리파이 비교 (시험 핵심!)

- 아두이노 (Arduino Uno)
  - 분류: 마이크로컨트롤러 (Atmega328)
  - OS: 없음 (운영체제 설치 불가, 부트로더만 내장)
  - 실행: 단일 프로그램만 실행 (멀티태스킹 불가)
  - 개발: PC에서 Sketch 프로그램으로 작성 후 업로드
- 라즈베리파이 (Raspberry Pi 4B)
  - 분류: 64비트 싱글보드 컴퓨터 (BCM2711)
  - OS: 있음 (라즈베리파이 OS, 데비안, 우분투 등)
  - 실행: 멀티태스킹 가능 (여러 응용프로그램 동시 실행)
  - 개발: 라즈베리파이 자체에서 파이썬, C/C++, Java 등 다양하게 직접 개발

3. 라즈베리파이 4B 주요 하드웨어 명칭 및 용도

- CPU/GPU SoC: 프로그램 실행 및 그래픽 처리 통합 칩 (BCM2711)
- GPIO 40핀: 외부 센서, LED, 모터 제어용 다용도 입출력 포트
- 마이크로 HDMI: 4K 듀얼 디스플레이 지원 (2개 탑재)
- CSI 포트: 라즈베리파이 전용 카메라(PiCam) 연결 단자
- DSI 포트: 라즈베리파이 전용 LCD 디스플레이 연결 단자
- 전원 단자: USB C-Type (5V)
- 메인 저장소: Micro SD 카드 (OS 및 파일 저장)

4. 주요 특징 및 용도

- 이름의 유래: Pi는 Python(파이썬)의 줄임말 (교육용 언어로 채택)
- Headless (헤드리스): 모니터/키보드/마우스 없이 네트워크로만 연결되어 작동하는 형태
- 응용 분야: 데스크톱 PC, 서버(웹/파일/NAS), 임베디드 컴퓨터(자율주행 RC카, 스마트 미러 등)
- 전원 관리 주의: 전원을 끌 때는 C타입 단자를 자주 뺐다 껴지 말고 멀티탭 전원을 분리

CHAPTER 02

[2주차 요약] 무선 통신 가능한 라즈베리파이 만들기

1. 무선 인터넷 준비 & 핫스팟 활용

- 무선 인터넷 연결 필수 정보: SSID (네트워크 이름) 및 비밀번호
- 무선 연결 방법 종류: 무선 공유기, 노트북 핫스팟(강력 추천), 스마트폰 핫스팟
- 노트북 핫스팟의 장점: 외부 공유기 없이도 노트북이 무선 공유기 역할을 수행하여 라즈베리파이가 쉽게 사설 IP를 할당받음

2. Raspberry Pi Imager를 이용한 OS 설치 과정

- 메인 저장소: Micro SD 카드 (16GB 이상 권장)
- 주요 설정 항목 (사용자 지정 옵션):
  - 호스트 이름 (Hostname): 네트워크상에서 라즈베리파이를 식별하는 고유 이름 (예: h + 학번)
  - 사용자 계정: 기본 계정 (예: ID = pi, PW = pi)
  - Wi-Fi 설정: 부팅 시 자동 연결할 SSID 및 비밀번호 사전 입력
  - 원격 접근 (SSH): ★ 필수 활성화 (비밀번호 인증 선택) - 켜지 않으면 원격 접속 불가

3. 헤드리스 (Headless) 라즈베리파이 & IP 확인

- 개념: 모니터, 키보드, 마우스 없이 네트워크(SSH)를 통해 원격으로만 조작하는 환경
- IP 주소 확인 방법:
  - 모니터 연결 시: 오른쪽 상단 와이파이 아이콘에 마우스 올려 확인 (또는 터미널에 hostname -I)
  - 노트북 핫스팟 사용 시: 윈도우 '모바일 핫스팟' 설정 창의 '연결된 장치' 목록에서 IP 확인
  - 무선 공유기 사용 시: 'Advanced IP Scanner' 프로그램으로 네트워크 스캔하여 확인
- 원격 접속 도구: PuTTY (SSH 프로토콜, 포트 번호 22번 사용)

4. 네트워크 제어 명령어 (시험 단골★)

- netplan vs NetworkManager:
  - netplan: 네트워크 연결 계획서(/etc/netplan/*.yaml)를 다루는 서비스
  - NetworkManager: netplan 계획대로 실제 네트워킹을 실행하는 서비스
- nmcli (NetworkManager Command Line Interface): 네트워크 제어 명령어
- 주요 nmcli 명령어:
  - 연결 목록 확인: nmcli connection show
  - 무선 네트워크 추가 (우선순위 설정 포함):

```
sudo nmcli connection add type wifi ifname wlan0 con-name [이름] ssid [SSID] wifi-sec.key-mgmt wpa-psk wifi-sec.psk "[비밀번호]" connection.autoconnect yes connection.autoconnect-priority [우선순위숫자]
```

(※ 우선순위 값이 클수록 높은 우선순위를 가짐)

- 네트워크 우선순위 변경:

```
sudo nmcli connection modify "[연결이름]" connection.autoconnect-priority [숫자]
```

- 주요 네트워크 설정 파일 저장 위치:
  - netplan 기본 설정 파일: /etc/netplan/*.yaml
  - nmcli로 추가된 네트워크 정보 파일: /etc/NetworkManager/system-connections/

5. 라즈베리파이 종료 및 재부팅 명령어 (시험 필수!)

- 안전 종료 (전원 분리 전 필수): sudo halt (또는 sudo poweroff / sudo shutdown -h now)
  - 주의: 그냥 전원을 끄면 SD 카드 데이터/프로그램 손상 위험!
- 재부팅: sudo reboot

6. 시스템 업데이트 및 한글 설정 (참고)

- 시스템 업데이트: sudo apt-get update && sudo apt-get upgrade
- 한글 설정 툴 실행: sudo raspi-config (Localisation Options -> Locale -> ko_KR.UTF-8 선택)

CHAPTER 03

[3주차 요약] 라즈베리파이 운영체제 (리눅스 기초 & 핵심 명령어)

1. 리눅스 운영체제 개요 및 특징

- 리눅스 탄생: 1991년 9월, 헬싱키 대학교의 '리누스 토발즈(Linus Torvalds)'가 개발 및 공개
- 구조: 하드웨어 <-> 커널(Kernel) <-> 쉘(Shell) <-> 응용 프로그램 <-> 사용자
  - 커널: 시스템의 핵심 자원(CPU, 메모리, 장치 드라이버 등)을 관리
  - 쉘(Shell): 사용자의 명령을 입력받아 해석하고 실행해 주는 프로그램 (라즈베리파이는 기본적으로 'bash' 사용)
- 주요 특징: 오픈소스(공개 소프트웨어), 유닉스(UNIX) 호환, 높은 이식성, 다중 사용자(Multi-user) 및 다중 프로세스(Multi-tasking) 지원

2. 쉘 프롬프트 구조 (시험 단골★)

- 예시: pi@h2471290:~/ch03 $
  - `pi` : 로그인한 사용자 ID
  - `h2471290` : 호스트 이름 (컴퓨터 이름)
  - `~/ch03` : 현재 디렉터리 위치 (~는 사용자 홈 디렉터리 /home/pi)
  - `$` : 일반 사용자 쉘 기호 (#은 최고 관리자 root 쉘 기호)

3. 리눅스 사용자 및 권한 관리

- 사용자 유형:
  - 루트 사용자 (root): 최고 권한(Super user), UID=0, 모든 파일/디렉터리 수정 및 삭제 가능
  - 일반 사용자 (예: pi, kitae 등): 자신의 홈 디렉터리 위주 작업, 권한이 부여된 영역만 접근 가능
- sudo 명령어: Superuser Do의 약자로, 일반 사용자가 root(수퍼유저) 권한을 일시적으로 위임받아 명령을 실행할 때 사용
- 계정 관리 주요 파일 (/etc 디렉터리 내):
  - `/etc/passwd` : 모든 사용자 계정 정보 저장 (ID, UID, GID, 홈디렉터리, 로그인쉘 등 7가지 정보 포함)
- 주요 계정 관리 명령어:
  - `useradd -m -c "설명" [계정명]` : 새로운 사용자 계정 생성 (-m: 홈 디렉터리 생성, -c: 설명 추가)
  - `passwd [계정명]` : 비밀번호 설정/변경
  - `userdel -r [계정명]` : 사용자 삭제 (-r: 홈 디렉터리 및 메일 스풀까지 완전 삭제)
  - `who / users / whoami` : 현재 접속 중인 사용자 정보 확인

4. 파일 시스템 & 경로

- 최상위 디렉터리: 루트(/)
- 경로 표현 방식:
  - 절대 경로: 루트(/)부터 시작하는 전체 경로 (예: /home/pi/ch03)
  - `상대 경로: 현재 위치를 기준으로 한 경로 (.` : 현재 디렉터리, .. : 부모 디렉터리, ~ : 홈 디렉터리)

5. vi 편집기 주요 모드 및 명령 (시험 필수★)

- 실행: vi [파일명] (예: vi hello.c)
- 모드 전환:
  - [명령 모드] -> [입력 모드]: i (커서 위치 삽입), a (커서 다음 삽입), o (아래에 빈 줄 생성하며 입력)
  - [입력 모드] -> [명령 모드]: ESC 키
- 명령 모드 주요 편집 키:
  - `x` : 한 글자 삭제
  - `dd` : 한 줄 전체 삭제
  - `:wq!` : 저장하고 강제 종료 (:w 저장, :q 종료)
  - `:q!` : 저장하지 않고 강제 종료
- C 언어 컴파일: gcc hello.c -> 컴파일 결과로 실행 파일 a.out 생성 -> 실행: ./a.out

6. 파일 및 디렉터리 제어 명령어

- `pwd` : 현재 작업 디렉터리의 절대 경로 출력
- `ls` : 파일 및 디렉터리 목록 출력 (-l: 상세 정보, -a: 숨김 파일 포함 전체 보기)
- `mkdir / cd` : 디렉터리 생성 / 이동
- `cp [원본] [복사본]` : 파일 복사
- `mv [원본] [이동/변경경로]` : 파일 이동 또는 이름 변경
- `rm` : 파일 삭제 (-r: 하위 디렉터리까지 일괄 삭제)
- `cat / more` : 파일 내용 출력 (cat: 전체 출력, more: 페이지 단위 출력)
- `tree` : 디렉터리 구조를 트리 모양으로 출력

7. 파일 속성 및 권한 변경 (chmod★)

- ls -l 출력 예시: -rwxr-xr-x 1 pi pi 8108 hello.c
  - 맨 앞 1글자 (타입): - (일반 파일), d (디렉터리), l (링크)
  - 권한 9자리 (3자리씩): [소유자(u)][그룹(g)][기타사용자(o)] -> r(읽기:4), w(쓰기:2), x(실행:1)
- 권한 변경 (chmod):
  - 8진수 방식: chmod 644 hello.c (rw-r--r--) / chmod 755 hello.c (rwxr-xr-x)
  - 기호 방식: chmod u+w [파일] (소유자 쓰기권한 추가), chmod o-x [파일] (기타사용자 실행권한 제거)

8. 프로세스 관리 및 유용한 명령

- 프로세스(Process): 실행 중인 프로그램 (운영체제가 유일한 PID 부여)
- 포그라운드(FG) vs 백그라운드(BG):
  - FG: 키보드 입력을 독점하는 상태
  - BG: 키보드 권한 없이 뒤에서 실행되는 상태 (명령어 끝에 & 붙임: ./a.out &)
- 단축키 및 제어 명령어:
  - `Ctrl + C` : 포그라운드 프로세스 강제 종료
  - `Ctrl + Z` : 포그라운드 프로세스 일시 정지 (Stop)
  - `jobs` : 정지/백그라운드 작업 목록 확인
  - `fg / bg` : 프로세스를 포그라운드로 전환 / 백그라운드로 전환
  - `ps -ef` : 시스템 전체 프로세스 상세 목록 조회
  - `kill -9 [PID]` : 해당 PID의 프로세스 강제 종료 (-9 옵션은 강제 종료 신호)

9. 검색, 파이프, 히스토리, 환경변수

- `grep [문자열] [파일명]` : 파일 내에서 특정 문자열이 포함된 라인 검색 (예: grep -n main *.c)
- `파이프 (|)` : 앞 명령의 출력을 뒤 명령의 입력으로 연결 (예: ps -ef | grep bash)
- `history` : 사용자가 입력했던 명령어 히스토리 조회 (!히스토리번호 로 재실행, !! 바로 전 명령 재실행)
- 패키지 관리 (apt-get):
  - sudo apt-get update (목록 업데이트)
  - sudo apt-get upgrade (실제 프로그램 최신화)
  - sudo apt-get install [패키지명] (설치)
  - sudo apt-get remove [패키지명] (제거, 설정유지) / purge (완전 제거)
- 환경변수 (PATH★):
  - `echo $PATH` : 실행 파일을 검색하는 경로 목록 확인
  - ./a.out으로 실행해야 하는 이유: 현재 디렉터리(.)가 PATH 환경변수에 등록되어 있지 않기 때문
  - PATH에 현재 디렉터리 추가: export PATH=.:$PATH

CHAPTER 04

[4주차 요약] 파이썬 기초 및 라즈베리파이 실습 핵심 정리

1. 파이썬 언어의 특징 및 실행 방식

- 개발자: 1991년 귀도 반 로섬(Guido van Rossum) 개발
- 주요 특징: 인터프리터 언어(컴파일 없이 바로 해석/실행), 플랫폼 독립적, 동적 타입(Dynamically Typed) 스크립트 언어, 객체지향 언어
- 파이썬 실행 방식 2가지:
  - 대화식 실행 방식 (Interactive): 터미널에 python 입력 후 한 줄씩 직접 실행 (>>> 프롬프트, Ctrl+D로 종료)
  - 파일 단위 실행 방식: vi 편집기 등으로 작성된 .py 파이썬 파일 실행 (예: python area.py)

2. 식별자, 주석, 문장 구분 및 입출력

- 식별자(Identifier) 규칙:
  - 첫 글자: 알파벳(A-Z, a-z), 언더스코어(_), $ 사용 가능 (숫자로 시작 불가능)
  - 대소문자 구분 (myHome과 myhome은 서로 다른 변수)
  - if, for, while 등 예약어(키워드) 사용 불가
- 주석문: 한 줄 주석(#), 여러 줄 주석(""" ... """)
- 문장 구분: 한 줄에 여러 문장을 쓸 때는 세미콜론(;) 필수, 한 줄에 한 문장이면 세미콜론 생략 가능
- 출력 print():
  - `print("A", end="; ")`: end 옵션으로 줄바꿈 대신 구분자 지정 가능
  - 포맷 출력: print("%d개의 %s %4.2f" % (3, msg, sum)) (%d: 정수, %s: 문자열, %f: 실수)
- 입력 input():
  - input()은 기본적으로 문자열(str)로 입력받음
  - 정수/실수 입력 시 형변환 필수: int(input()), float(input())

3. 데이터 타입, 리터럴, 형변환

- 주요 데이터 타입: 숫자(정수, 실수), 논리(True, False - 대문자 주의), 문자열, 객체 레퍼런스, None(값이 없음을 의미, NULL과 유사)
- 리터럴 표기법:
  - 8진수: 0o15 (숫자 0과 알파벳 o로 시작, 10진수로 13)
  - 16진수: 0x15 (숫자 0과 알파벳 x로 시작, 10진수로 21)
  - 논리형: True, False (소문자 true/false 사용 시 에러)
- 형변환 함수:
  - int(), float(), str(), bool(), chr()(ASCII 번호->문자), ord()(문자->ASCII 번호)
  - bool(0)은 False, bool("True")나 bool(0이 아닌 숫자)는 True
  - int("10.5")처럼 실수 형태의 문자열을 정수로 바꾸려 하면 에러 발생 (float("10.5") 후 int() 처리 필요)

4. 연산자와 식

- 산술 연산자: +, -, *, / (실수 나누기), // (정수 나누기/몫), % (나머지)
  - 예: 11 / 4 = 2.75, 11 // 4 = 2, 11 % 4 = 3
- 대입 연산자: =, +=, -=, *=, /=, //=, %= 등
- 비교 연산자: >, >=, <, <=, ==, != (결과는 True/False)
- 논리 연산자: and, or, not
- 삼항 조건 연산자: 변수 = 값1 if (조건식) else 값2 (예: big = x if (x > y) else y)
- 문자열 연산: + 연산자로 문자열 연결 ("abc" + str(23) -> "abc23")

5. 조건문과 반복문

- 조건문 (if, if-else, if-elif-else):
  - 콜론(:)으로 블록 시작을 나타내며, 동일한 크기의 들여쓰기(Indent) 필수
- 반복문 (for, while):
  - `range(start, stop, step)` : range(5) -> 0~4, range(2, 5) -> 2~4, range(10, 0, -2) -> 10, 8, 6, 4, 2
  - 문자열 반복: for c in "Falling": 문자열의 문자를 하나씩 순회
  - break문: 반복문을 완전히 벗어남
  - continue문: 이하 문장을 건너뛰고 다음 반복 순서로 진행

6. 컨테이너 객체 (리스트, 튜플, 딕셔너리★)

- 리스트(List): 대괄호 [] 사용, 원소 수정/삭제/추가 가능, 서로 다른 타입 저장 가능
  - 주요 함수: list.append(x), list.insert(index, x), list.pop(), list.remove(x), list.sort(reverse=True)
- 튜플(Tuple): 소괄호 () 사용, 읽기 전용 컨테이너로 원소 수정 및 삭제 불가능
- 딕셔너리(Dictionary): 중괄호 {} 사용, <Key, Value> 쌍 구조
  - 주요 함수: dict.keys(), dict.values(), dict.items(), dict.get(key) (없는 키 조회 시 None 반환)

7. 함수 (Function)

- 정의: def 함수이름(매개변수1, 매개변수2):
- 매개변수 디폴트 값: def adder(a, b=20): (호출 시 b를 생략하면 20 적용)
- 명명된 매개변수: adder(b=30, a=20) 처럼 매개변수 이름을 지정하여 순서와 상관없이 호출 가능
- 전역변수(Global) vs 지역변수(Local):
  - 함수 내부에서 전역변수 값을 수정/쓰기 하려면 반드시 global 키워드 선언 필요 (global x)
  - global 선언 없이 전역변수 x의 값을 수정하려고 x = x + 10 시 UnboundLocalError 발생

8. 모듈 (Module)

- 개념: 하나의 파이썬 파일(.py 파일)
- 임포트: import 모듈이름 / from 모듈이름 import 함수명
- __name__ 전역변수 (시험 단골★):
  - 모듈이 단독(직접)으로 실행될 때: __name__ == '__main__'
  - 모듈이 다른 파일에 import되어 실행될 때: __name__ == '모듈이름'

9. 파일 처리 (File I/O)

- 파일 열기: file = open("파일명", "모드")
  - 모드: "r"(읽기), "w"(쓰기, 기존 내용 덮어씀/삭제), "a"(추가 쓰기), "x"(새 파일 생성 쓰기, 파일 있으면 에러)
- 파일 읽기/쓰기:
  - `file.write(str)` : 파일에 문자열 쓰기
  - `file.readlines()` : 모든 줄을 읽어 문자열 리스트로 반환
  - `file.close()` : 사용 완료 후 파일 닫기

CHAPTER 05

[5주차 라즈베리파이 카메라 활용]

4.1 카메라 소개

- 라즈베리파이용 카메라는 파이 카메라와 USB 카메라 2종류가 있음
- 파이 카메라는 CSI 소켓에 연결되는 전용 소형 카메라임
- USB 카메라는 라즈베리파이의 USB 포트에 연결하여 다양한 해상도(SD, HD, FHD, 4K)를 지원함

4.2 VNC를 활용한 라즈베리파이에 원격 접속

- `VNC(Virtual Network Computing)` : 원격에서 동일한 GUI 화면을 보며 마우스와 키보드를 사용하는 기술
- raspi-config 명령어를 통해 라즈베리파이의 VNC 서버(Wayvnc/X11)를 활성화
- PC에서 RealVNC Viewer를 설치한 후 라즈베리파이 IP 주소 및 계정 정보를 입력하여 원격 접속 및 데스크톱 GUI 활용

4.3 USB 카메라 제어

- lsusb 명령어로 라즈베리파이에 연결된 USB 카메라 장치 정상 인식 여부 확인
- fswebcam 패키지 설치: sudo apt-get install fswebcam
- fswebcam image.jpg 명령을 통해 USB 카메라로 사진을 촬영하고 파일로 저장
- 다양한 옵션 활용: -r 640x480 (해상도 지정), --rotate 180 (180도 회전) 등

4.4 파이썬 가상환경

- 독립된 파이썬 실행 환경을 제공하여 패키지/라이브러리 간의 버전 충돌 문제 방지
- 가상환경 생성: python -m venv myenv
- 가상환경 활성화: source myenv/bin/activate (프롬프트 앞에 (myenv) 표시)
- 가상환경 비활성화: deactivate
- pip 명령어를 통해 가상환경 내 패키지 설치(install), 삭제(uninstall), 목록 확인(list) 관리

4.5 OpenCV로 카메라 제어

- `OpenCV(Open Source Computer Vision Library)` : 컴퓨터 비전 및 기계 학습 라이브러리
- 가상환경 내 설치: pip install opencv-python (또는 opencv-contrib-python)
- cv2.VideoCapture(0, cv2.CAP_V4L) 객체를 생성하여 카메라 프레임 캡처
- cv2.imread() / cv2.imshow() / cv2.imwrite() / cv2.destroyAllWindows() 주요 함수 활용
- VideoCapture의 버퍼 동작 원리 이해: read() 호출 속도와 버퍼 공급 속도 불일치 시 카메라 지연(Delay) 발생
- camera.grab() 함수를 반복 호출하여 버퍼에 쌓인 과거 프레임을 모두 버리고 현재 시점의 프레임을 획득하여 딜레이 해결

4.6 카메라로 얼굴 인식

- Haar Cascades: 머신러닝 기반의 물체 및 얼굴 인식 알고리즘
- GitHub에서 학습 모델 xml 파일 다운로드 (haarcascade_frontalface_default.xml)
- cv2.CascadeClassifier()로 검출 객체 생성
- BGR 컬러 이미지를 cv2.cvtColor()로 흑백(GrayScale) 변환 후 classifier.detectMultiScale()을 실행하여 얼굴 좌표(x, y, w, h) 배열 획득
- cv2.rectangle()을 이용해 검출된 faces 영역에 노란색(0, 255, 255) 사각형 박스를 그려 실시간 추적 화면 표시 (4-6.py 실습)

[트러블슈팅]

- AttributeError: module 'cv2' has no attribute 'CascadeClassifier'
- 원인: 클래스명 대소문자 오타 또는 가상환경 내 opencv 패키지 설치 꼬임
- 해결: pip uninstall opencv-python opencv-contrib-python -y 실행 후 pip install opencv-contrib-python 재설치

CHAPTER 06

[6장 라즈베리파이로 웹서버 만들기 - 핵심 요약]

1. 웹 서버 vs 웹 응용 서버(WAS)

- 웹 서버: HTML, CSS, 이미지 등 '정적 콘텐츠'를 브라우저에 전송
- 웹 응용 서버(WAS): DB 연동 및 로직 실행을 통해 '동적 콘텐츠' 생성
- 웹 서버 프레임워크: 웹 서버 + WAS 기능을 합친 라이브러리 (예: Python Flask, Django)

2. 플라스크(Flask) 개요 및 기본 동작

- 파이썬 기반의 소형 마이크로 웹 서버 프레임워크
- `URL 라우팅 테이블([URL : 실행함수])`을 바탕으로 요청에 맞는 파이썬 함수 호출
- 기본 실행: app.run(host='0.0.0.0', port=8080, debug=True)
  - host='0.0.0.0': 외부의 모든 IP 접속 허용
  - debug=True: 소스 수정 시 자동 재실행 및 디버깅 정보 출력

3. 플라스크 필수 디렉터리 구조

- ~/ch06/

```
├── static/      : 정적 파일 저장 공간 (HTML, CSS, JS, 이미지 등) -> 자동 라우팅 처리
└── templates/   : 동적 rendering에 사용될 템플릿 HTML 파일 저장 공간
```

4. URL 라우팅 & Variable

- `@app.route('/path/')` : URL과 함수 매핑
- 슬래시(/) 주의점:
  - @app.route('/kitae/') 지정 시: /kitae 로 접근해도 308 리다이렉트로 정상 연결
  - @app.route('/kitae') 지정 시: /kitae/ 로 접근하면 404 Not Found 에러 발생!
- Variable 활용: URL 일부를 함수의 매개변수로 전달
  - 예: @app.route('/withdraw/<int:money>/') -> 함수 def bank(money): 로 정수값 전달
  - 변환 타입: int, float, path, uuid

5. 템플릿 HTML과 render_template()

- `render_template('hello.html', name=val)` : templates 폴더 내 HTML에 스크립트 변수를 전달하여 동적 HTML 생성(렌더링)
- 템플릿 스크립트 문법:
  - `{{ 변수/수식 }}` : 값 출력
  - `{% 스크립트 %}` : 제어문 (if, for 등)
  - `{% endif %}, {% endfor %}` : 블록 종료

6. 데이터 전송 방식 (GET vs POST)

- GET 방식:
  - 데이터를 URL 헤더(Query String: ?key=value)에 노출하여 전송
  - 파이썬 수신: request.args['key']
- POST 방식:
  - 데이터를 요청 바디(Body)에 포함하여 전송
  - 라우팅 설정: @app.route('/calc', methods=['POST'])
  - 파이썬 수신: request.form['key']

7. 원격 카메라 & CCTV 스트리밍

- 동작 흐름: 브라우저 요청 -> cctvapp.py 실행 -> camera.py(take_picture)로 사진 촬영 -> ./static/cctv.jpg 저장 -> cctv.html 렌더링
- 실시간 스트리밍 구현:
  - HTML 내 <meta http-equiv="refresh" content="0.5; url={%print(request.path)%}"> 추가
  - 0.5초마다 자동으로 페이지를 재요청하여 연속 촬영 이미지를 받아옴으로써 CCTV 스트리밍 효과 구현

CHAPTER 07

[7장 메시지 통신과 MQTT - 핵심 요약]

1. MQTT 개념 및 특징

- MQTT (Message Queuing Telemetry Transport): 경량 메시지 기반 통신 프로토콜 (TCP/IP 기반, 기본 포트: 1883)
- 발행/구독(Publish/Subscribe) 방식:
  - Publisher (발행자): 특정 '토픽(Topic)'으로 메시지를 브로커에 전송
  - Subscriber (구독자): 관심 있는 '토픽'을 브로커에 등록하고, 해당 메시지가 오면 수신
  - Broker (중계자): Publisher와 Subscriber 사이에서 메시지를 중계 (1:1, 1:N, N:M 가능)
- 특징: 송신자와 수신자가 직접 연결되지 않고 브로커를 경유함. 열악한 네트워크 환경/IoT에 최적화.

2. MQTT 패킷, 토픽 & QoS

- 토픽 (Topic): 메시지를 구분하는 계층화된 문자열 (예: "room/1/temp")
- QoS (Quality of Service) - 메시지 전송 품질 (Publisher/Subscriber ↔ Broker 사이):
  - Level 0 (At most once): 최하, 확인 없이 최대 1회 전송 (손실 가능)
  - Level 1 (At least once): 보통, 수신 확인 응답을 받음 (중복 수신 가능)
  - Level 2 (Exactly once): 최고, 4단계 핸드셰이크로 정확히 1회 전송 보장

3. 라즈베리파이 Mosquitto 브로커 설정

- 설치: sudo apt install -y mosquitto mosquitto-clients
- 설정 파일 (mos.conf):

```
listener 1883        # 외부 접속 허용
allow_anonymous true # 익명 사용자 허용
```

- 실행 명령: mosquitto -v -c mos.conf (-v: 과정 상세 출력)
- 기본 서비스 비활성화: sudo systemctl disable/stop mosquitto

4. 파이썬 paho-mqtt 라이브러리 (v2 기준)

- 설치: pip install paho-mqtt
- 객체 생성: client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
- 주요 함수:
  - `connect(ip, port=1883)` : 블록킹 연결
  - `connect_async(ip, port=1883)` : 넌블록킹 연결 (on_connect 콜백 필수)
  - `subscribe("topic")` : 토픽 구독
  - `publish("topic", "message")` : 메시지 발행
  - `disconnect()` : 브로커 연결 종료
- 주요 콜백 속성:
  - `on_connect = 함수명` : 브로커 연결 성공 시 실행
  - `on_message = 함수명` : 메시지 도착 시 실행 (msg.topic, msg.payload)
  - on_subscribe / on_publish / on_disconnect

5. 메시지 루프 (Message Loop)

- `loop()` : 메시지가 있을 때 1번만 처리하는 블록킹 함수 (while True와 조합)
- `loop_forever()` : 무한히 메시지 루프 실행 (블록킹, 이후 코드 실행 안 됨)
- `loop_start() / loop_stop()` : 별도 스레드를 생성하여 배경에서 루프 실행 (메인 코드와 병렬 처리 가능)
- Ping 메커니즘: 메시지 루프 사용 시 keepalive 시간 내에 ping을 자동으로 보내 연결 유지

6. 라즈베리파이 ↔ 윈도우 교차 통신 응용

- 동일 네트워크(Wi-Fi/핫스팟) 연결 필수
- 라즈베리파이(Publisher/Subscriber) + 윈도우(Subscriber/Publisher) + Mosquitto 브로커(라즈베리파이)
- 이미지/바이너리 전송:
  - cv2.imencode('.jpg', frame)[1].tobytes()로 이미지 바이트화 후 publish
  - 수신 측에서는 open(filename, "wb")로 파일 저장